การทำงานกับ **Tailwind CSS, React Router และ Context API**

Function 2.1 - The Personal Branding SPA

---

1. วัตถุประสงค์

1. ฝึกการจัด Layout และทำ Responsive ด้วย **Tailwind CSS**
2. เข้าใจการทำงานของ **React Router** ในการสร้าง Single Page Application (SPA)
3. ประยุกต์ใช้ **Context API** เพื่อจัดการ Global State (Dark/Light Mode)

หมายเหตุ Final Project จะต้องเป็นแอปที่มีฟีเจอร์เทียบเท่าฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist ด้านล่างนี้ได้ผ่านทุกข้อ

2. โครงสร้าง

จัดโครงสร้างไฟล์ดังนี้:

Plaintext

```
src/
├── context/
│   └── ThemeContext.jsx    # เก็บ Global State สำหรับ Dark Mode
├── layouts/
│   └── MainLayout.jsx      # โครงสร้างหลัก (Navbar, Footer, Outlet)
├── pages/
│   ├── Home.jsx            # หน้าแรก
│   ├── Projects.jsx        # หน้ารวมผลงาน
│   └── Contact.jsx         # หน้าติดต่อ
└── App.jsx                 # แหล่งรวม Routing
```

---

3. ขั้นตอนการทำ

1. **Setup Project:**
   - สร้างโปรเจกต์ด้วย Vite: `npm create vite@latest my-portfolio -- --template react`
   - ติดตั้ง Tailwind CSS ตามเอกสารทางการ
   - ติดตั้ง Router: `npm install react-router-dom`
2. **จัดการ Routing (ใน App.jsx):**
   - ใช้ `<BrowserRouter>` หุ้ม `<ThemeProvider>`
   - ใช้ `<Routes>` และ `<Route>` โดยให้ `MainLayout` เป็นตัวแม่และมีหน้าอื่นๆ เป็นตัวลูก

3. **สร้างหน้า Pages:**
    - **Home.jsx:** แสดง Hero Section (ชื่อและคำโปรย) ใช้ Tailwind ทำตัวอักษรใหญ่ๆ และปุ่ม Call to Action
    - **Projects.jsx:** สร้าง Array ของงาน (เช่น `{id: 1, name: 'Web A'}`) แล้วใช้ `.map()` เพื่อสร้าง Card แสดงผล (Grid 3 คอลัมน์บนจอใหญ่)
    - **Contact.jsx:** สร้างฟอร์มง่ายๆ (ชื่อ, อีเมล, ข้อความ) จัดการสไตล์ด้วย Tailwind `focus:ring-2`
4. **Dark Mode Integration:**
    - ตรวจสอบว่าใน `tailwind.config.js` มีการเพิ่ม `darkMode: 'class'` แล้ว
    - ใช้คลาส `dark:...` ในการกำหนดสีเมื่ออยู่ในโหมดมืด (เช่น `bg-white dark:bg-black`)

Checklist Function 2.1

ลำดับ	รายการตรวจสอบ (Requirement)		
C1	**Routing:** สลับหน้า Home, Projects, Contact ได้โดยไม่รีเฟรชหน้าเว็บ		
C2	**Global State:** กดปุ่มสลับ Dark/Light Mode แล้วเปลี่ยนสีทั้งแอปได้จริง		
C3	**Responsive:** หน้า Projects แสดงผล 1 คอลัมน์ในมือถือ และ 3 ในคอมพิวเตอร์		
C4	**Tailwind Utility:** มีการใช้ Hover Effect และ Transition ที่สวยงาม		
C5	**Structure:** จัดโครงสร้างไฟล์แยก Context, Layout, Page ชัดเจน		

<u>หมายเหตุ</u> Final Project จะต้องเป็นแอปที่มีฟีเจอร์<u>เทียบเท่า</u>ฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist นี้ได้ผ่านทุกข้อ

คำถาม

1. หากเราไม่ใช้ `Context API` ในการทำ Dark Mode แต่ใช้การส่ง `Props` แทน จะเกิดปัญหาอะไรขึ้นเมื่อแอปพลิเคชันมีขนาดใหญ่ (Component ซ้อนกัน 10 ชั้น)?
2. ทำไมเราถึงต้องใส่ `<Outlet />` ไว้ใน `MainLayout` ? หากลบทิ้งจะเกิดอะไรขึ้น?

การจัดการ **Dynamic Routing** และการจัดการข้อมูลภายในหน้าเว็บ (Local State)

Function 2.2 - Product Discovery & Filtering

---

1. วัตถุประสงค์

1. เข้าใจการใช้งาน **Dynamic Routing** และการส่งค่าผ่าน URL (**URL Parameters**)
2. ฝึกการใช้ Hook `useParams` เพื่อดึงข้อมูลมาแสดงผลตาม ID
3. ฝึกการใช้ Hook `useNavigate` เพื่อเปลี่ยนหน้าด้วยคำสั่ง JavaScript
4. เรียนรู้การทำระบบ **Search & Filter** เบื้องต้นโดยใช้ Local State

<u>หมายเหตุ</u> Final Project จะต้องเป็นแอปที่มีฟีเจอร์<u>เทียบเท่า</u>ฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist ด้านล่างนี้ได้ผ่านทุกข้อ

2. โครงสร้าง

Plaintext

```
src/
├── data/
│   └── products.js        # ไฟล์เก็บข้อมูลจำลอง (Mock Data)
├── pages/
│   ├── Catalog.jsx        # หน้าแสดงรายการสินค้าทั้งหมด + ช่องค้นหา
│   └── ProductDetail.jsx  # หน้าแสดงรายละเอียดสินค้าทีละชิ้น
└── App.jsx                # การตั้งค่า Routes
```

3. ขั้นตอนการทำ

1. **ตั้งค่า Routing ใน `App.jsx` :**
   - กำหนด Path `/` ให้แสดงคอมโพเนนต์ `Catalog`
   - กำหนด Path `/product/:id` ให้แสดงคอมโพเนนต์ `ProductDetail` (เน้นย้ำการใส่เครื่องหมาย `:`)
2. **ทำระบบ Search:**
   - ใช้ `useState` เพื่อผูกค่ากับ Input (Controlled Component)
   - ใช้ฟังก์ชัน `.filter()` และ `.includes()` ในการกรองข้อมูลแบบ Real-time
3. **เชื่อมโยงข้อมูล:**
   - ในหน้า `Catalog` ต้องใช้ `<Link>` ส่งตัวแปร `id` ไปที่ URL

- ในหน้า `ProductDetail` ต้องใช้ `useParams` ดึง ID มาค้นหาข้อมูลใน Array

4. **ตกแต่งด้วย Tailwind:**
   - ใช้คลาส `grid` , `gap` , `shadow` , `transition` เพื่อให้หน้าเว็บดูเป็นมืออาชีพ

Checklist Function 2.2

ลำดับ	รายการตรวจสอบ (Requirement)		
C1	**Dynamic Routing:** เข้าหน้า `/product/1` หรือ `/product/2` แล้วแสดงข้อมูลต่างกันได้		
C2	**Search Logic:** เมื่อพิมพ์ชื่อสินค้าในช่องค้นหา รายการสินค้าต้องลดลงตามที่กรองไว้		
C3	**useNavigate:** มีปุ่ม "Back" ที่ใช้งานได้จริงโดยสั่งงานผ่านโค้ด (ไม่ใช่แค่ Link)		
C4	**NotFound Handling:** หากกรอก ID ที่ไม่มีในข้อมูล (เช่น `/product/99`) ต้องแสดงข้อความเตือน		
C5	**Tailwind Layout:** มีการจัด Grid สินค้าที่รองรับการแสดงผลบนมือถือ (Responsive)		

<u>หมายเหตุ</u> Final Project จะต้องเป็นแอปที่มีฟีเจอร์<u>เทียบเท่า</u>ฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist นี้ได้ผ่านทุกข้อ

คำถาม

1. หากเราไม่ใส่เครื่องหมาย `:` หน้าคำว่า `id` ในไฟล์ `App.jsx` (เช่น `path="/product/id"`) ผลลัพธ์ที่ได้จะเป็นอย่างไร?
2. การใช้ `useNavigate(-1)` มีความแตกต่างจากการใช้ `<Link to="/">` อย่างไรในแง่ของประสบการณ์ผู้ใช้งาน (UX)?

จัดการกับลอจิกการคำนวณที่ซับซ้อนและ State ที่มีความสัมพันธ์กันหลายจุดผ่าน **Zustand**

Function 2.3 - The Advanced Shopping Cart

---

1. วัตถุประสงค์

1. ฝึกการใช้ **Zustand** ในการจัดการ Global State แทนการใช้ Context API
2. ฝึกการเขียน **Complex Action Logic** (การเพิ่มสินค้าลงตะกร้าโดยเช็คสินค้าซ้ำ)
3. ฝึกการทำ **Derived State** (การคำนวณยอดรวม ภาษี และค่าขนส่งจากข้อมูลใน Store)
4. ฝึกการทำ **Side Drawer UI** ด้วย Tailwind CSS

หมายเหตุ Final Project จะต้องเป็นแอปที่มีฟีเจอร์เทียบเท่าฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist ด้านล่างนี้ได้ผ่านทุกข้อ

---

2. โครงสร้าง

Plaintext

```
src/
├── store/
│   └── useCartStore.js     # หัวใจหลัก: คลังเก็บข้อมูลตะกร้าและฟังก์ชันจัดการ
├── components/
│   ├── ProductCard.jsx     # ส่วนแสดงผลสินค้ารายชิ้น
│   └── CartDrawer.jsx      # แถบด้านข้างแสดงรายการในตะกร้าและยอดรวม
└── App.jsx                 # หน้าหลักที่รวมทุกอย่างเข้าด้วยกัน
```

3. ขั้นตอนการทำ (Step-by-Step)

1. **สร้าง Store:** เขียนไฟล์ `useCartStore.js` ตาม Template โดยเพิ่มฟังก์ชันอัปเดตจำนวนสินค้า (เพิ่ม/ลด +/-) ให้สมบูรณ์
2. **Product List:** ในหน้าหลัก ให้แสดงรายการสินค้าจำลอง 4-5 อย่าง โดยแต่ละอันมีปุ่ม "Add to Cart" ที่เรียกใช้แอคชันจาก Store
3. **Cart Drawer:** * สร้าง UI แถบข้างที่สามารถเปิด/ปิดได้ (ใช้สถานะ `isOpen` ใน Local State)
   - แสดงรายการสินค้าในตะกร้า: ชื่อ, ราคา, จำนวน, และปุ่มลบ
4. **Summary Logic:** * แสดงยอดรวมราคาสินค้าทั้งหมด
   - คำนวณภาษี 7% (VAT)
   - เพิ่มค่าจัดส่ง 100 บาท (แต่ถ้าตะกร้าว่าง ให้เป็น 0)
5. **Navbar Badge:** แสดงตัวเลขจำนวนชนิดสินค้าในตะกร้าบนมุมขวาของไอคอนตะกร้าสินค้าที่ Navbar

Checklist Function 2.3

ลำดับ	รายการตรวจสอบ (Requirement)		
C1	**Zustand Usage:** มีการสร้างและเรียกใช้ Store ได้ถูกต้องตามหลักการ		
C2	**Quantity Logic:** เมื่อเพิ่มสินค้าซ้ำ จำนวน (Quantity) ต้องเพิ่มขึ้น ไม่ใช่เพิ่มแถวใหม่		
C3	**Calculation Accuracy:** ยอดรวม ภาษี และค่าส่ง ต้องคำนวณได้ถูกต้องแม่นยำ		
C4	**Interactive UI:** กดปุ่มลบ หรือปุ่มล้างตะกร้าแล้วข้อมูลต้องอัปเดตทันทีทุกจุด		
C5	**Tailwind Mastery:** การทำ Drawer มีแอนิเมชันเลื่อนเข้า-ออก หรือ Overlay ที่สวยงาม		

หมายเหตุ Final Project จะต้องเป็นแอปที่มีฟีเจอร์เทียบเท่าฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist นี้ได้ผ่านทุกข้อ

คำถาม

1. ทำไมเราถึงเลือกใช้ Zustand แทนที่จะใช้ `useState` ในหน้า `App.jsx` แล้วส่ง Props ลงไปให้ Navbar และ Drawer? (อธิบายเรื่องประสิทธิภาพ)
2. หากเราต้องการให้ข้อมูลในตะกร้าไม่หายไปเมื่อผู้ใช้กด Refresh หน้าจอ (Browser Refresh) นิสิตจะนำแนวคิด "Middleware" ของ Zustand (เช่น `persist`) มาช่วยได้อย่างไร?

การจัดการข้อมูลที่ต้อง คงอยู่ (Persistence) ข้ามหน้า และการใช้โครงสร้าง Routing ที่ซับซ้อนแบบ Nested Routes

---

Function 2.4 - Multi-step Job Application Portal

---

1. วัตถุประสงค์

1. ฝึกการใช้ **Nested Routing** และคอมโพเนนต์ `<Outlet />` เพื่อสร้าง Layout ของฟอร์มที่คงที่
2. จัดการข้อมูลฟอร์มขนาดใหญ่ที่แบ่งเป็นหลายหน้าโดยใช้ **Zustand** เป็นตัวเก็บข้อมูลส่วนกลาง
3. เรียนรู้การทำ **Guarded Navigation** เพื่อป้องกันไม่ให้ผู้ใช้ข้ามขั้นตอนหากยังกรอกข้อมูลไม่ครบ
4. ฝึกการทำ **Data Summary** ดึงข้อมูลจากหลายแหล่งใน State มาแสดงผลรวมกัน

หมายเหตุ Final Project จะต้องเป็นแอปที่มีฟีเจอร์เทียบเท่าฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist ด้านล่างนี้ได้ผ่านทุกข้อ

2. โครงสร้าง

Plaintext

```
src/
├── store/
│   └── useFormStore.js     # เก็บข้อมูลผู้สมัครงานจากทุก Step
├── layouts/
│   └── FormLayout.jsx      # Layout หลักของฟอร์ม (Progress Bar + Outlet)
├── pages/
│   ├── Step1.jsx           # ข้อมูลส่วนตัว
│   ├── Step2.jsx           # ประสบการณ์การทำงาน
│   └── Review.jsx          # ตรวจสอบข้อมูลทั้งหมด
└── App.jsx                 # Nested Route Configuration
```

3. ขั้นตอนการทำ (Step-by-Step)

1. **Configure Nested Routes:** ใน `App.jsx` ให้สร้าง Route `/apply` ที่มี `element={<FormLayout />}` และมีลูกเป็น `step-1`, `step-2`, และ `review`
2. **Step 1 (Personal Info):** * สร้าง Input สำหรับชื่อ นามสกุล อีเมล
   - เมื่อกด "Next" ให้เรียก `updateFormData` แล้วใช้ `useNavigate` ไปยัง `step-2`
3. **Step 2 (Experience):** * สร้าง Textarea สำหรับระบุประสบการณ์และทักษะ
   - เมื่อกด "Next" ให้ตรวจสอบก่อนว่าข้อมูลหน้าแรก (Email) มีอยู่จริงใน Store หรือไม่ (Simple Guard)
4. **Review Page:** * ดึง `formData` ทั้งหมดจาก Store มาแสดงในรูปแบบตารางหรือรายการ

- มีปุ่ม "Confirm & Submit" ซึ่งเมื่อกดแล้วให้แสดง Alert ยินดีด้วย และเรียก `resetForm`

5. **Tailwind Styling:** ตกแต่งปุ่ม "Next" และ "Back" ให้สวยงาม และทำ Responsive ให้ฟอร์มอ่านง่ายบนมือถือ

Checklist Function 2.4

ลำดับ	รายการตรวจสอบ (Requirement)		
C1	**Nested Routing:** ใช้งาน `<Outlet />` ได้ถูกต้อง และ Layout ไม่กระพริบเวลาเปลี่ยน Step	[ ]	
C2	**State Persistence:** กรอกข้อมูลหน้า 1 แล้วกดไปหน้า 2 แล้วกด Back กลับมา ข้อมูลต้องไม่หาย	[ ]	
C3	**Progress Tracking:** Progress Bar ด้านบนต้องขยับตาม Step ที่ผู้ใช้อยู่จริง	[ ]	
C4	**Form Validation:** มีการเช็คเบื้องต้น (เช่น ถ้าไม่กรอกชื่อ ห้ามกด Next)	[ ]	
C5	**Data Review:** หน้าสุดท้ายสามารถรวบรวมข้อมูลจากทุก Step มาแสดงผลได้ถูกต้อง	[ ]	

<u>หมายเหตุ</u> Final Project จะต้องเป็นแอปที่มีฟีเจอร์<u>เทียบเท่า</u>ฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist นี้ได้ผ่านทุกข้อ

คำถาม

1. การใช้ **Nested Routes** มีข้อดีอย่างไรต่อการออกแบบ UI ที่มีส่วนประกอบร่วมกัน (เช่น Sidebar หรือ Progress Bar)?
2. หากความต้องการเปลี่ยนเป็น "ห้ามผู้ใช้เข้าหน้า Review โดยตรงผ่าน URL ถ้ายังไม่ผ่าน Step 1" จะใช้วิธีใดในการดักจับ (Guard) สถานการณ์นี้?

การจัดการสิทธิ์เข้าถึง (RBAC), การทำ Protected Routes, และการจัดการ State ที่ซิงโครไนซ์กับ URL

Function 2.5 - Collaborative Dashboard with Role-based Access

---

1. วัตถุประสงค์

1. พัฒนาระบบ **Protected Routes** เพื่อจำกัดการเข้าถึงหน้าเว็บตามสิทธิ์ (Role-based Access Control)
2. จัดการสถานะการเข้าสู่ระบบจำลอง (Auth Simulation) ด้วย **Zustand**
3. ฝึกการใช้ `useSearchParams` เพื่อเก็บสถานะการกรองข้อมูล (Filter) ไว้บน URL (URL-as-State)
4. ฝึกการทำ **Conditional Rendering** ขั้นสูงตามระดับสิทธิ์ของผู้ใช้งาน

<u>หมายเหตุ</u> Final Project จะต้องเป็นแอปที่มีฟีเจอร์<u>เทียบเท่า</u>ฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist ด้านล่างนี้ได้ผ่านทุกข้อ

2. โครงสร้าง

Plaintext

```
src/
├── store/
│   └── useAuthStore.js      # เก็บข้อมูล User และ Role (admin/user)
├── components/
│   └── ProtectedRoute.jsx   # ส่วนห่อหุ้มสำหรับเช็คสิทธิ์ก่อนเข้าหน้าเว็บ
├── pages/
│   ├── Login.jsx            # หน้าเลือกสิทธิ์เพื่อจำลองการ Login
│   ├── Dashboard.jsx        # หน้าแสดงงาน พร้อมระบบ Filter ผ่าน URL
│   └── AdminSettings.jsx    # หน้าเฉพาะ Admin เท่านั้น
└── App.jsx                  # การตั้งค่า Route Guard ทั้งหมด
```

3. ขั้นตอนการทำ (Step-by-Step)

1. **Auth Simulation:** ในหน้า `Login.jsx` สร้างปุ่ม 2 ปุ่มคือ "Login as Admin" และ "Login as User" เมื่อคลิกให้เซตข้อมูลลงใน `useAuthStore`
2. **Route Guarding:** ใน `App.jsx` ให้หุ้ม Route ดังนี้:
   - `/dashboard` : หุ้มด้วย `ProtectedRoute` (อนุญาตทั้ง admin และ user)
   - `/admin-settings` : หุ้มด้วย `ProtectedRoute` (อนุญาตเฉพาะ `allowedRoles={['admin']}`)

3. **URL Search Params:** ในหน้า Dashboard เมื่อกด Filter ข้อมูล URL ต้องเปลี่ยน และเมื่อผู้ใช้กด Refresh หน้าเว็บ ข้อมูล Filter ใน Dropdown ต้องคงเดิม (ดึงมาจาก URL)
4. **Role-based UI:** ในหน้า Dashboard หาก User เป็น `admin` ให้แสดงปุ่ม "Delete Project" แต่หากเป็น `user` ปกติ ให้ซ่อนปุ่มนั้นไว้
5. **Layout Switching:** สร้าง Sidebar ที่จะแสดงเมนูไม่เหมือนกันตาม Role ที่ Login อยู่

Checklist Function 2.5

ลำดับ	รายการตรวจสอบ (Requirement)		
1	**Auth Logic:** สามารถจำลองการ Login/Logout และเก็บสถานะลง Zustand ได้จริง	[ ]	
2	**Role Guard:** User ทั่วไปไม่สามารถเข้าหน้า Admin ผ่านการพิมพ์ URL ตรงๆ ได้	[ ]	
3	**URL Sync:** เมื่อกดเปลี่ยน Filter ค่าบน URL ต้องเปลี่ยนตาม (เช่น `?status=done`)	[ ]	
4	**Deep Linking:** เมื่อก๊อปปี้ URL ที่มี Param ไปเปิดแท็บใหม่ ต้องแสดงผลตาม Filter นั้นทันที	[ ]	
5	**Conditional UI:** หน้าตาเว็บเปลี่ยนไปตาม Role อย่างชัดเจน (เช่น เมนูพิเศษสำหรับ Admin)	[ ]	

<u>หมายเหตุ</u> Final Project จะต้องเป็นแอปที่มีฟีเจอร์<u>เทียบเท่า</u>ฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist นี้ได้ผ่านทุกข้อ

คำถาม

1. เพราะเหตุใดการเก็บสถานะ Filter ไว้บน URL (`searchParams`) ถึงดีกว่าการเก็บไว้ใน `useState` ปกติในแง่ของ User Experience?
2. หากต้องการให้ระบบจำได้ว่า User คนเดิม Login อยู่แม้จะปิด Browser ไปแล้ว จะเลือกใช้วิธีใด?